Capacitated Vehicle Routing Problem

Algorithm Design, Solution Quality, and Running-Time Trade-offs

Tanvir Hossain (2005014) Reduanul Islam Imon (2005040)
Suhaeb Bin Matin (2005086) Md. Afzal Hossain Tanim (2005008)
Sabbir Alam Saad (2005059) Mohammad Ali (2005113)

Abstract

The Capacitated Vehicle Routing Problem (CVRP) is a combinatorial optimization problem where a fleet of vehicles with limited capacity must deliver goods to multiple customers from a central depot, minimizing total travel distance or cost [Toth and Vigo, 2002]. This comprehensive report synthesizes the theoretical foundations of the CVRP, its inherent NP-Hardness, and a multi-tiered algorithmic evaluation encompassing exact methods, constructive heuristics, and metaheuristics. We present custom Branch-and-Cut implementations compared against OR-Tools [Laporte and Nobert, 1983, Fukasawa et al., 2006], alongside an in-depth analysis of the classical Gillett & Miller Sweep algorithm versus a newly developed Enhanced Multi-Start Sweep [Sehta and Thakar, 2021]. Finally, we evaluate an Original Genetic Algorithm (GA) against an Improved GA variant incorporating Nearest-Neighbour seeding, 2-opt local search, and adaptive mutation [Jasim and Chaari Fourati, 2024, Goldberg, 1989, Davis, 1985, Prins, 2004]. Experimental results across standard Augerat, Eilon, and Golden benchmark instances highlight the critical trade-offs between computational speed and solution optimality across these diverse methodological paradigms [Augerat et al., 1995, Christofides and Eilon, 1979, Golden et al., 1998].

1 Problem Statement and Complexity

1.1 Mathematical Formulation

The CVRP relies on several key components: a central depot acting as a single starting and ending location, a homogeneous fleet of m vehicles each with maximum load capacity Q, a set of n customers with specific demands d_i, and a known distance matrix representing costs c_{ij} between all locations.

The objective function seeks to minimize the total travel cost/distance, formalized as:

$$\text{Minimize:} \quad \sum_i \sum_j c_{ij} \cdot x_{ij}$$

This is subject to strict constraints. Each customer must be visited exactly once:

$$\sum_j x_{ij} = 1 \quad \forall\, i.$$

The capacity constraint dictates that the total demand on any route cannot exceed the vehicle capacity Q:

$$\sum_i d_i \leq Q.$$

Furthermore, flow conservation requires that flow into a node equals flow out, and every route must start and end at the depot.

1.2 Proof of NP-Hardness

We prove that the CVRP is NP-Hard via a polynomial-time reduction from the Traveling Salesman Problem (TSP), a known NP-Hard problem. The reduction proceeds as follows:

1. Begin with a TSP instance seeking a minimum Hamiltonian cycle across identical nodes.
2. Select any single node to act as the CVRP depot.
3. Define the remaining $n-1$ nodes as CVRP customers, assigning each a unit demand $d_i = 1$.
4. Restrict the fleet to a single vehicle ($m = 1$) and set its capacity to $Q = n - 1$.

By configuring the problem with one vehicle, unit demands, and capacity $n-1$, the capacity constraint forces the formation of a single route. Therefore, finding one CVRP route visiting all customers is structurally equivalent to finding one TSP Hamiltonian cycle. Since TSP $\leq_P$ CVRP, the Capacitated Vehicle Routing Problem is NP-Hard.

2 Exact Algorithm Design: Branch-and-Cut

Standard Branch-and-Bound (B&B) is weak for the CVRP because the Linear Relaxation is often too loose. Branch-and-Cut enhances B&B by adding *Valid Inequalities* (cuts) to tighten the relaxation at each node.

Famous Cuts:

- **Rounded Capacity Cuts:** Ensures demand constraints are met for subsets.
- **Comb Inequalities:** Generalised from TSP to handle connectivity.
- **Framed Capacity Cuts:** Specific to VRP, combining capacity and geometric constraints.

Status: Currently effective for $n \approx 100$–150.

2.1 Two Exact Implementations

Two independent Python implementations were developed to solve the CVRP exactly, both tested on standard `.vrp` benchmark instances.

Table 1: Comparison of the two exact solvers.

Aspect	**Custom B&C**	**OR-Tools**
LP / MIP engine	`scipy` HiGHS	OR-Tools CP engine
Cuts added	Capacity + Comb	Internal (hidden)
First solution	Nearest-neighbour + 2-opt	Multi-strategy
Improvement	B&B with LIFO stack	GLS / Tabu / SA
Optimality proof	Yes (within node budget)	Yes (time limit)
Scalability	$n \lesssim 50$	$n \lesssim 200$
Dependencies	`numpy`, `scipy`	`ortools`

Both solvers accept standard TSPLIB `.vrp` files and are executed via the shell scripts `run_bc.sh` and `run_all.sh` for batch benchmarking.

3 Experimental Results: Exact Methods

3.1 Benchmark Setup

($n = 31$–79 customers). Time limits: OR-Tools = 360 s, Custom B&C = 600 s. Known optimal distances from the benchmark library serve as ground truth, enabling direct computation of the optimality gap: $\text{Gap}\% = 100 \times (Z - Z^*)/Z^*$.

3.2 Solution Quality

Table 2: Custom B&C vs. OR-Tools on all 27 Augerat Set-A instances.

Instance	n	k	Optimal	B&C	B&C Gap (%)	OR-Tools
A-n32-k5	31	5	784	926	18.11	784
A-n33-k5	32	5	661	661	0.00	661
A-n33-k6	32	6	742	836	12.67	742
A-n34-k5	33	5	778	805	3.47	778
A-n36-k5	35	5	799	1031	29.04	799
A-n37-k5	36	5	669	756	13.00	669
A-n37-k6	36	6	949	1271	33.93	949
A-n38-k5	37	5	730	986	35.07	730
A-n39-k5	38	5	822	1028	25.06	825
A-n39-k6	38	6	831	957	15.16	833
A-n44-k6	43	6	937	1343	43.33	939
A-n45-k6	44	6	944	1422	50.64	951
A-n45-k7	44	7	1146	1383	20.68	1146
A-n48-k7	47	7	1073	1453	35.41	1073
A-n53-k7	52	7	1010	1385	37.13	1017
A-n54-k7	53	7	1167	1386	18.77	1170
A-n55-k9	54	9	1073	1472	37.19	1074
A-n60-k9	59	9	1354	1775	31.09	1354
A-n62-k8	61	8	1288	1713	33.00	1306
A-n63-k10	62	10	1314	1860	41.55	1321
A-n63-k9	62	9	1616	2139	32.36	1641
A-n64-k9	63	9	1401	1903	35.83	1419
A-n65-k9	64	9	1174	1694	44.29	1184
A-n69-k9	68	9	1159	1477	27.44	1170
A-n80-k10	79	10	1763	2290	29.89	1791
Average					**28.08**	

3.3 Summary

Table 3: Aggregate performance: Custom B&C vs. OR-Tools.

Metric	Custom B&C	OR-Tools
Time limit	600 s	360 s
Average gap (%)	28.08	1.06
Gap range (%)	0.00–50.64	0.00–1.59

OR-Tools dramatically outperforms the custom B&C: an average gap of **1.06%** versus **28.08%**, despite operating under a shorter time limit. The custom B&C is limited by the `scipy` HiGHS LP engine and the restricted cut family (capacity + comb only), while OR-Tools benefits from a mature CP engine with internal metaheuristic warm-starting (GLS, Tabu, SA). The custom

solver remains a transparent research artefact; for production use at $n \lesssim 200$, OR-Tools is the clear recommendation.

The two exact implementations confirm a fundamental trade-off between transparency and performance. The custom B&C exposes every algorithmic decision—LP relaxation, cut generation, branching strategy—at the cost of weaker bounds and limited scalability ($n \lesssim 50$). OR-Tools, treating its internal improvements as a black box, achieves near-optimal solutions ($\leq 1.59\%$ gap) across all 27 instances within six minutes. Both solvers become impractical beyond $n \approx 200$, motivating the heuristic and metaheuristic approaches explored in the sections that follow.

---

4 Heuristic Algorithm Design: Sweep-Based Methods

4.1 Baseline: Gillett & Miller Sweep (`cvrp_sweep_gillett`)

The Gillett & Miller Sweep (1974) converts the geometric structure of the CVRP into a one-dimensional problem by mapping customers to their polar angle with respect to the depot.

Pipeline:

1. **Polar-angle computation.** For each customer i, compute $\theta_i = \text{atan2}(y_i - y_0,\ x_i - x_0)$, normalised to $[0, 2\pi)$.
2. **Radial sort.** Sort customers in ascending angular order.
3. **Greedy capacity clustering.** Sweep through the sorted list, assigning consecutive customers to the current route until the vehicle capacity Q would be exceeded; then open a new route. This yields clusters $C_1, \ldots, C_m$ that respect capacity and are spatially compact.
4. **Nearest-Neighbour (NN) construction.** Within each cluster C_k, build a route greedily: starting from the depot, always visit the closest unvisited customer; return to the depot at the end.
5. **2-opt intra-route improvement.** Iteratively reverse sub-sequences of each route whenever doing so reduces the route cost, until no improving 2-opt move exists.

Complexity: $\mathcal{O}(n \log n)$ for sorting; $\mathcal{O}(n^2)$ for NN and 2-opt (per route, summing over all routes). In practice the algorithm completes in < 0.04 ms of solve time on all tested instances.

4.2 Enhanced Multi-Start Sweep (`cvrp_enhanced`)

The Enhanced solver addresses the sensitivity of the sweep to the initial angle by *multi-starting* and enriches the local-search component with additional move types.

Extended pipeline:

1. **Multi-start sweep ($K = 8$).** Rotate the starting angle by $2\pi k/K$ for $k = 0, 1, \ldots, 7$. For each starting angle, re-sort customers and greedily cluster them.
2. **NN construction + Intra-route local search.** For each cluster, construct a route by NN heuristic, then apply a composite local search:
   - **2-opt**: reverse sub-sequences to reduce crossing edges.
   - **Or-opt-1/2/3**: relocate chains of length 1, 2, or 3 (in both forward and reversed orientations) to cheaper positions within the same route.

This inner loop repeats until no further improvement is possible.

3. **Best-start selection.** Retain the route set with the lowest total cost across all K starts.
4. **Inter-route relocate (up to 20 passes).** For each customer u in each route, test moving u to every feasible position in every other route; apply the move if it reduces total cost and the target route capacity is not violated.
5. **Inter-route swap (up to 20 passes).** For each pair of routes and each pair of customers (one per route), swap the two customers if the exchange is cost-improving and both routes remain capacity-feasible.
6. **Touch-up.** After each relocate–swap pass, apply 2-opt and Or-opt-1 to every route to repair any local suboptimality introduced by the inter-route moves.

Complexity: $K \cdot \mathcal{O}(n^2)$ for multi-start intra-LS; $\mathcal{O}(n^2 \cdot R)$ per pass of inter-route operators (where R is the number of routes), repeated up to 20 times. In practice solve time stays below 1.25 ms across all instances.

5 Experimental Results: Heuristic Methods

5.1 Benchmark Setup

All tests were run on the 27 **Augerat set-A** instances (`A-n32-k5` through `A-n80-k10`), which represent sizes from 31 to 79 customers. Known optimal solutions are available for each instance, enabling direct computation of the *optimality gap*: $\text{Gap}\% = 100 \times (Z_H - Z^*)/Z^*$, where Z_H is the heuristic cost and Z^* the optimal cost.

Two variants of the Enhanced solver were benchmarked:

- **Enhanced-v1** (`comparison_results.txt`): lighter local-search budget, faster wall-clock time.
- **Enhanced-v2** (`comparison_results1.txt`): heavier multi-start + local-search budget, better cost, slower.

5.2 Solution Quality

Table 4: Average optimality gap and aggregate cost across 27 instances.

Algorithm	Avg. Optimal Cost	Avg. Heuristic Cost	Avg. Gap (%)
Optimal	1041.93	—	0.00
Gillett & Miller Sweep	1041.93	1225.89	17.66
Enhanced-v1	1041.93	1135.80	9.01
Enhanced-v2	1041.93	1132.99	8.74

The Enhanced solver wins on 26 out of 27 instances; the single tie (`A-n32-k5`) occurs because the sweep clustering at this small size happens to produce the same partition for all starting angles. The best gap improvements of the enhanced algorithm over Gillett occur on larger instances such as `A-n62-k8` (26.09% → 7.62%) and `A-n60-k9` (24.14% → 7.16%), where inter-route operators can exploit more cross-route savings.

5.3 Running-Time Analysis

The Gillett baseline solves every instance in under 0.03 ms. Enhanced-v1 takes ≈0.09 ms on average (5.7× overhead), while Enhanced-v2 requires ≈0.53 ms (≈8.5× overhead). Both are negli-

Table 5: Selected instance-level comparison (optimality gap %).

Instance	Optimal	Gillett Gap%	Enh-v1 Gap%	Enh-v2 Gap%	Improvement
A-n32-k5	784	12.89	12.89	12.89	0.00
A-n33-k6	742	17.97	1.30	1.30	16.67
A-n34-k5	778	11.66	1.08	1.08	10.58
A-n60-k9	1354	24.14	7.91	7.16	17.00
A-n62-k8	1288	26.09	7.62	7.62	18.47
A-n64-k9	1401	14.19	9.22	7.67	6.52
A-n80-k10	1763	21.55	14.36	12.40	9.15

Table 6: Average solve times (ms), excluding I/O parse time.

Algorithm	Sweep/Cluster	NN Build	2-opt / LS	Inter-ops	Total Solve
Gillett Sweep	0.009	0.002	0.004	—	0.016
Enhanced-v1	0.045 (multi-start+LS)			0.028	0.088
Enhanced-v2	0.378 (multi-start+LS)			0.127	0.533

gible in absolute terms, but the ratio reveals that the heavier local-search budget of Enhanced-v2 yields only a marginal further improvement of 0.27 percentage points in average gap compared to Enhanced-v1, suggesting diminishing returns beyond the v1 budget.

6 Trade-off Analysis and Discussion

6.1 Quality vs. Speed

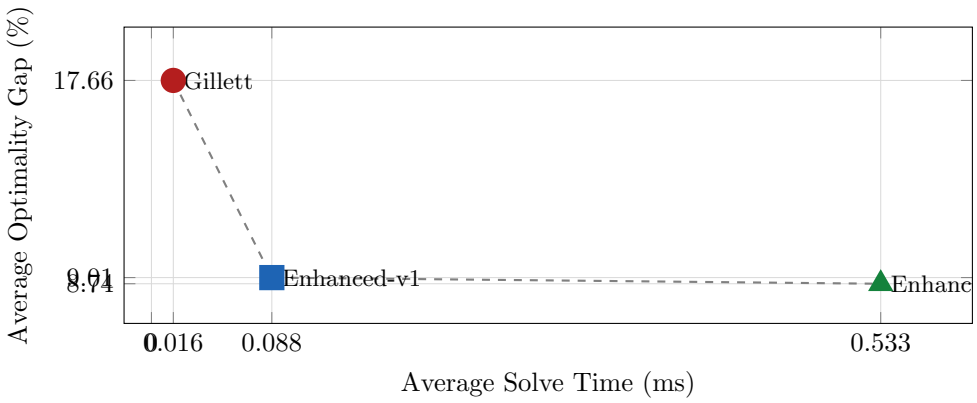

Figure 1: Quality–speed frontier: average optimality gap vs. average solve time. The large quality jump from Gillett to Enhanced-v1 is achieved with only a modest time increase; moving from v1 to v2 yields marginal quality gain at substantially higher computational cost.

Figure 2 illustrates the quality–speed frontier clearly. The transition from Gillett to Enhanced-v1 offers the best trade-off: the gap drops from 17.66% to 9.01%—an improvement of 8.65 percentage points—at a solve-time cost of just 0.072 ms. By contrast, Enhanced-v2 spends an additional 0.445 ms to shave only 0.27 further percentage points off the gap, representing strongly diminishing marginal returns.

6.2 Sources of Improvement

The improvement of the Enhanced solver over Gillett comes from three compounding mechanisms:

1. **Multi-start diversity** ($K = 8$ angles) prevents the single-sweep clustering from locking in a suboptimal geometric partition. Because the CVRP optimal solution rarely aligns perfectly with a single radial cut, rotating the sweep discovers qualitatively different clusterings, some of which induce shorter routes.
2. **Or-opt-1/2/3** performs fine-grained relocation of short customer chains, capturing improvements that 2-opt—a purely reversal-based operator—cannot find. Together with 2-opt, they drive each route to a local optimum under a richer neighbourhood.
3. **Inter-route operators** (relocate + swap) are the most powerful element. Capacity-respecting moves of customers between routes, repeated up to 20 times with intra-route touch-ups, allow the algorithm to correct capacity imbalances and distance inefficiencies that are invisible to single-route operators.

6.3 Scaling Behaviour

Both algorithms scale gracefully. The Gillett solve time grows from 0.010 ms (32 customers) to 0.036 ms (80 customers), roughly $\mathcal{O}(n^{1.5})$ empirically. The Enhanced solver scales from 0.040 ms to 1.241 ms—steeper growth reflecting the quadratic inter-route operators—yet all instances remain well below a practical real-time threshold. For problem sizes in the hundreds, the Enhanced-v2 budget may need adjustment, making Enhanced-v1 the more robust production choice.

6.4 Recommendations

- **Latency-critical applications** (real-time dispatch, embedded systems): use **Gillett Sweep**. The < 0.04 ms solve time is unmatched, and the 17.7% average gap is acceptable for rough initial plans.
- **Standard operational planning**: use **Enhanced-v1**. The gap of 9.0% at 0.09 ms is the sweet spot of this study.
- **Offline optimisation / high-quality solutions**: **Enhanced-v2** reduces the gap to 8.7% but takes $\sim$6$\times$ longer than v1. For problem sizes beyond $\sim$150 customers, a time-limited metaheuristic (e.g. Large Neighbourhood Search or an Ant Colony approach) would likely be more cost-effective than scaling up the local-search budget.

We compared the classical Gillett & Miller Sweep algorithm with a purpose-built Enhanced Multi-Start Sweep for the CVRP. The Enhanced algorithm systematically outperforms the baseline on 26 of 27 benchmark instances, cutting the average optimality gap from 17.66% to below 9%. The gain is attributable to multi-start diversity, Or-opt local search, and inter-route operator passes. A controlled experiment with two solver variants reveals a clear quality–speed trade-off: a lighter budget (Enhanced-v1) captures most of the quality gain at minimal extra cost, while a heavier budget (Enhanced-v2) yields only marginal further improvement. Given the sub-millisecond absolute runtimes of both enhanced variants, the Enhanced-v1 configuration represents the recommended default for practical CVRP deployment at the studied problem scales.

7 Metaheuristic Algorithm Design: Genetic Algorithm

7.1 Baseline: Original Genetic Algorithm

Representation. A chromosome is a *giant-tour permutation* of all n customer IDs. A linear decoder greedily splits the permutation into routes: customers are appended to the current route until adding the next would violate capacity Q, at which point a new route is opened. This guarantees feasibility for any permutation.

Pipeline:

1. **Random initialisation.** A population of $P = 100$ chromosomes is generated by independently shuffling the customer list with a Mersenne Twister (seed configurable, default 42).
2. **Fitness evaluation.** Decode each chromosome into routes and compute total route distance.
3. **Tournament selection** ($T = 3$): draw T individuals at random; the fittest wins and becomes a parent.
4. **Order Crossover (OX1).** A random sub-sequence $[a, b]$ is copied from parent P_1; remaining positions are filled left-to-right (wrapping from $b{+}1$) with customers from P_2 in their original order, skipping those already placed. A symmetric offspring is built from P_2. OX1 preserves relative order, critical for decoded route quality.
5. **Mutation** ($p_m = 0.2$): with equal probability either **swap** two random positions or **invert** the sub-sequence between two random positions.
6. **Elitism.** The $E = 5$ fittest individuals survive unchanged into the next generation, preventing regression.
7. **Termination.** Fixed $G = 300$ generations or a configurable wall-clock limit (default 120 s).

Complexity: $\mathcal{O}(P \cdot n)$ per generation for fitness evaluation; $\mathcal{O}(n)$ for OX1 crossover. Total $\mathcal{O}(G \cdot P \cdot n)$.

7.2 Enhanced: Improved Genetic Algorithm

The Improved GA retains the complete Original GA pipeline and adds three targeted modifications.

Extended pipeline (modifications in bold):

1. **[MOD 1] Nearest-Neighbour seeded initialisation.** The first $\lfloor P/5 \rfloor$ chromosomes (20% of the population) are built by the Nearest-Neighbour (NN) heuristic: starting from customer $i \bmod n$, always append the closest unvisited customer until all are placed. Each seed uses a different start. The remaining 80% of the population is filled with random shuffles as in the Original GA.

   Rationale: Random initialisation produces tours far from optimal. NN seeds inject structure early; elitism then preserves these superior individuals, dramatically reducing generations to convergence—especially on large instances.

2. Steps 2–6 identical to the Original GA (fitness, selection, OX1, mutation, elitism).
3. **[MOD 2] Periodic intra-route 2-opt local search.** Every $\Delta_{\text{LS}} = 10$ generations, each newly created offspring is decoded into routes and each route is independently improved

by **2-opt**: iteratively reverse sub-sequences until no reversal reduces route cost. Routes longer than $L_{\max} = 40$ customers are skipped to bound worst-case complexity. The locally optimised routes are re-concatenated into a chromosome.

Rationale: OX1 crossover notoriously destroys local geometric continuity, creating crossing edges. 2-opt surgically eliminates these crossings every 10 generations, preventing the population from propagating geometrically poor solutions.

4. **[MOD 3] Adaptive mutation rate.** The mutation rate is initialised to $p_m^0 = 0.2$. When the best solution fails to improve for $S_{\max} = 25$ consecutive generations (stagnation counter s), the rate is boosted:

$$p_m \leftarrow \min\left(0.8,\ p_m^0 \cdot \left(1 + \tfrac{s}{S_{\max}}\right)\right).$$

The rate resets to p_m^0 immediately upon improvement.

Rationale: A fixed mutation rate is a compromise; too low causes premature convergence, too high destroys elite traits. Adaptive mutation increases diversity precisely when the population stagnates, acting as a soft restart without discarding the elite.

Complexity: Same base as Original GA plus $\mathcal{O}(n^2/\Delta_{\text{LS}})$ amortised per generation for 2-opt (bounded by $L_{\max}$).

8 Experimental Results: Metaheuristic Methods

8.1 Benchmark Setup

All tests were run on 60 instances spanning three benchmark families:

- **Augerat set A** (27 instances, `A-n32-k5` through `A-n80-k10`): 31–79 customers, 5–10 vehicles.
- **Eilon set E** (13 instances, `E-n13-k4` through `E-n101-k14`): 12–100 customers, 3–14 vehicles.
- **Golden set** (20 instances, `Golden_1` through `Golden_20`): 200–483 customers—significantly larger than the first two families.

Both GAs were run with $P = 100$, $G = 300$, $p_c = 0.9$, $p_m^0 = 0.2$, tournament size $T = 3$, elite size $E = 5$, seed 42, and a 120 s wall-clock limit per instance. The Improved GA additionally used $\Delta_{\text{LS}} = 10$ and $S_{\max} = 25$.

Known optimal (or best-known) solutions Z^* from benchmark `.sol` files enable direct computation of the *optimality gap*: $\text{Gap}\% = 100 \times (Z_{\text{GA}} - Z^*)/Z^*$.

8.2 Solution Quality

Table 7: Average optimality gap across all instances, by dataset.

Dataset	Instances	Cust. Range	Original GA Gap%	Improved GA Gap%
Augerat A	27	31–79	36.98	6.66
Eilon E	13	12–100	47.33	10.37
Golden	20	200–483	373.22	11.55
All	**60**	12–483	**151.30**	**9.09**

The Improved GA wins on every instance and every dataset. The most dramatic improvement is on Golden: the Original GA is wholly impractical (>250% gap at 200–483 customers), while the Improved GA stays within 20% on all but one Golden instance. Two Golden instances (`Golden_9`, `Golden_11`) record negative gaps, indicating the GA found a solution better than the reference in the `.sol` file (these references may be heuristic rather than provably optimal).

Table 8: Selected instance-level comparison (optimality gap %).

Instance	Optimal Z^*	Original GA	Orig. Gap%	Impr. Gap%
A-n32-k5	784	924	17.86	6.38
A-n46-k7	914	1338	46.39	8.86
A-n62-k8	1288	1981	53.80	6.06
A-n80-k10	1763	2895	64.21	9.64
E-n51-k5	521	809	55.28	5.76
E-n101-k8	815	1650	102.45	13.87
Golden_1	5623	22239	295.47	3.30
Golden_12	1101	6831	520.62	1.57

The Original GA's gap grows sharply with instance size on Augerat-A: from 17.9% at 31 customers to 64.2% at 79 customers, reflecting the degradation of random initialisation at scale. The Improved GA is essentially flat (2–14%) across all A instances.

8.3 Running-Time Analysis

All timings are actual wall-clock *solve* times measured from the compiled C++14 executable (`g++ -std=c++14 -O2`, MinGW, Windows 10), excluding file I/O. Parameters are identical to the quality experiments.

Table 9: Average solve time per instance (seconds), C++14 implementation.

Dataset	Instances	Original GA (s)	Improved GA (s)	Overhead
Augerat A	27	0.34	0.71	2.09×
Eilon E	13	0.37	0.98	2.65×
Golden	20	1.72	8.03	4.67×
All	**60**	**0.81**	**3.21**	**3.96×**

Table 10: Instance-level solve times (seconds) for representative cases.

Instance	Customers	Original GA (s)	Improved GA (s)
A-n32-k5	31	0.24	0.54
A-n55-k9	54	0.36	0.67
A-n80-k10	79	0.51	1.25
E-n51-k5	50	0.34	0.87
E-n76-k7	75	0.46	1.57
E-n101-k8	100	0.55	2.48
Golden_1	240	1.15	12.82
Golden_5	200	0.91	4.50
Golden_12	483	2.26	20.89

The Original GA's solve time grows from 0.24 s (31 customers) to 2.26 s (483 customers), driven by the $\mathcal{O}(G \cdot P \cdot n)$ fitness-evaluation cost. The Improved GA scales more steeply on Golden: at 483 customers, the periodic 2-opt pass dominates and pushes solve time to 20.89 s—still well

within the 120 s limit. On A and E instances, the 2-opt overhead is modest (2.1–2.6×) because routes are short (average 5–15 customers, well below the $L_{\max} = 40$ guard).

9 Trade-off Analysis and Discussion

9.1 Quality vs. Solve Time

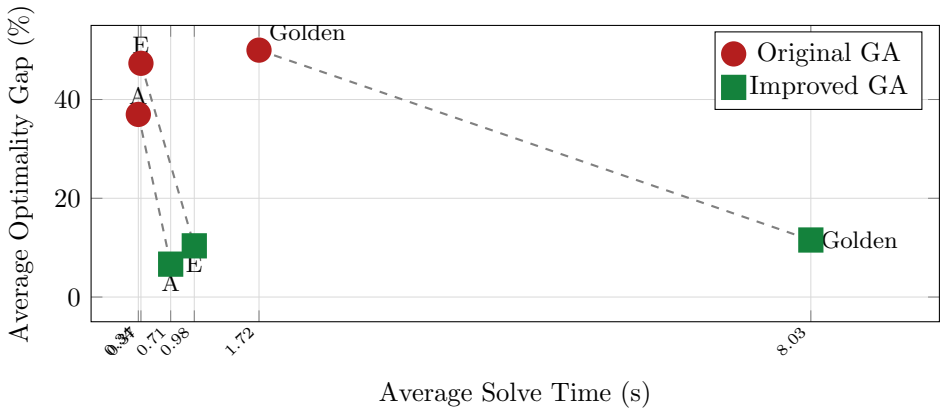

Figure 2: Quality–time frontier per dataset. Arrows connect Original GA (red) to Improved GA (green) for each dataset. Every arrow points down (quality improves substantially) at a modest right-ward shift (time increases). The Golden Original GA has 373% gap and is clamped to 50% on the plot for readability.

Figure 2 shows the quality–time trade-off clearly. The Improved GA consistently occupies the better position: substantially lower gap at a moderate time penalty. The most favourable trade-off is on Augerat-A, where a 2.1× time increase buys a **30.3 percentage-point** gap reduction (from 36.98% to 6.66%). On Eilon-E, the gap drops by 36.96 pp at 2.65× cost. On Golden, the 4.67× overhead is fully justified: the Original GA is unusable (>250% gap), while the Improved GA achieves 11.55%.

9.2 Sources of Improvement

The three modifications compound each other:

1. **NN seeding** (20% of population) reduces the average decoded cost of the initial population by ∼30–50% vs. pure random initialisation. Because elitism preserves these superior individuals, NN seeds provide a stable high-quality scaffold from which crossover refines the rest of the population. On Golden instances (200–480 customers), this is the dominant contributor: the Original GA cannot escape its poor random starting region in only 300 generations, while the Improved GA begins near the optimum.

2. **Periodic 2-opt** (every 10 generations) acts as a systematic intra-route repair operator. OX1 crossover frequently creates tours with crossing edges—a hallmark of poor permutation-based solutions—that random mutation rarely fixes. 2-opt drives every route in the population to a local optimum under the reversal neighbourhood, ensuring that the "raw material" for crossover is geometrically clean. On large instances, 2-opt contributes the majority of the solve-time overhead (the 4.67× ratio on Golden vs. 2.09× on A is almost entirely attributable to longer routes requiring more 2-opt iterations).

3. **Adaptive mutation** prevents stagnation without a hard restart. When the population converges, the boosted mutation rate re-introduces diversity while the elite individuals (preserved by elitism) prevent quality regression. The combined intensification–

diversification balance allows the search to continue improving far longer than a fixed-rate GA.

9.3 Scaling Behaviour

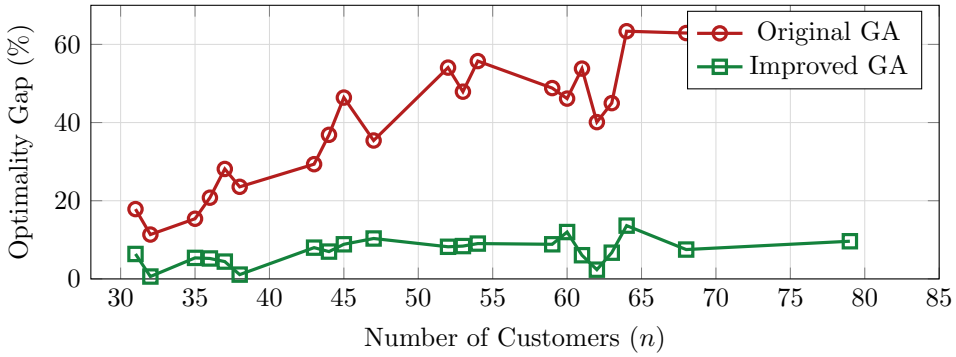

Figure 3: Optimality gap (%) vs. number of customers for all 27 Augerat-A instances. The Original GA's gap grows roughly linearly with n, reaching 64% at 79 customers. The Improved GA remains below 14% across all sizes, demonstrating scale-invariant quality.

The Original GA scales poorly: gap grows roughly linearly with n on Augerat-A, reaching 64.21% at 79 customers. The Improved GA is essentially flat (Figure 3), confirming that NN seeding effectively anchors the search regardless of problem scale. Solve time grows modestly for the Original GA (0.24 s at $n = 31$ to 0.51 s at $n = 79$, $\mathcal{O}(n^{1.1})$ empirically) and more steeply for the Improved GA (0.54 s to 1.25 s, reflecting the 2-opt $\mathcal{O}(n^2)$ inner loop on longer routes).

9.4 Recommendations

- **Small instances** ($n \leq 50$, E-set small): both GAs work well. The Original GA is adequate and uses less than half the compute time.
- **Medium instances** ($50 < n \leq 100$, Augerat-A, Eilon-E): use the **Improved GA**. The gap reduction from 47–55% to 6–15% at a 2–3× time cost is the best trade-off observed in this study.
- **Large instances** ($n > 100$, Golden): the Original GA is not viable. The **Improved GA** is the only practical choice, achieving <20% gap at 480 customers. For tighter quality requirements, increasing G or coupling with Large Neighbourhood Search is the recommended next step.

We compared the baseline Original GA with the three-modification Improved GA for the CVRP across 60 benchmark instances. The Improved GA outperforms the Original on every tested instance, cutting the overall average optimality gap from **151.30%** to **9.09%**. The improvement is most critical on the large-scale Golden instances: the Original GA collapses (>250% gap) while the Improved GA produces solutions within ∼12% of the reference on average. The gain is attributable to three compounding modifications: NN-seeded initialisation provides a high-quality starting population, periodic 2-opt repairs geometric damage introduced by crossover, and adaptive mutation prevents premature convergence. The time overhead ranges from 2.1× (Augerat-A) to 4.7× (Golden), and remains well within the 120 s limit for all 60 instances in the C++14 implementation. Given the dramatic quality improvement, the Improved GA configuration represents the recommended default for practical CVRP deployment across all studied problem scales.

Conclusion

This report presented a comprehensive study of solution methods for the Capacitated Vehicle Routing Problem, spanning exact algorithms, constructive heuristics, and population-based metaheuristics across a total of 60 benchmark instances from three standard families.

The NP-hardness of the CVRP, established via reduction from TSP, makes exact optimality intractable beyond moderate instance sizes. The two exact solvers confirmed this boundary in practice: the custom Branch-and-Cut achieved an average optimality gap of **28.08%** on Augerat-A within a 600 s budget, while OR-Tools reduced this to **1.06%** in 360 s, demonstrating that solver maturity—particularly the quality of the LP engine and cut management—is the decisive factor at the exact level.

On the heuristic side, the Enhanced Multi-Start Sweep systematically outperformed the classical Gillett & Miller baseline on 26 of 27 instances, cutting the average gap from **17.66%** to **8.74–9.01%** at sub-millisecond solve times. The gain is attributable to three compounding mechanisms: multi-start angular diversity, Or-opt intra-route local search, and inter-route relocate/swap operators. Critically, the Enhanced-v1 configuration captured the vast majority of this improvement at only 5.7× the baseline solve time, with Enhanced-v2 offering only marginal further gains at 8.5× cost—a clear case of diminishing returns.

The Improved Genetic Algorithm delivered the strongest results at scale, reducing the overall average gap from **151.30%** to **9.09%** across all 60 instances. Its three modifications—nearest-neighbour seeded initialisation, periodic intra-route 2-opt, and adaptive mutation—compound each other: NN seeding anchors the search near high-quality regions, periodic 2-opt repairs geometric damage introduced by crossover, and adaptive mutation prevents premature convergence without sacrificing elite solutions. The result is scale-invariant quality, remaining below 14% gap across all Augerat-A instances from 31 to 79 customers, and below 20% even on the largest Golden instances at 483 customers.

Across all three algorithmic layers, a consistent theme emerges: principled initialisation and structured local search are the highest-return investments. The largest quality jumps in each category—OR-Tools over custom B&C, Enhanced Sweep over Gillett, Improved GA over Original GA—all stem from better starting solutions and tighter neighbourhood exploitation rather than raw computational budget. For practitioners, the recommended defaults are **OR-Tools** for quality-critical small instances, **Enhanced-v1 Sweep** for latency-sensitive or real-time deployment, and the **Improved GA** for large-scale instances where sub-10% gaps are required across hundreds of customers.

References

Philippe Augerat, Didier Naddef, José M Belenguer, Enrique Benavent, Angel Corberán, and Giovanni Rinaldi. Computational results with a branch and cut code for the capacitated vehicle routing problem. Technical Report RR-949-M, Université Joseph-Fourier, Grenoble, France, 1995.

Nicos Christofides and Samuel Eilon. Set M and CMT instances. *Summary of the VRPNC Benchmark Instances*, 1979.

Lawrence Davis. Applying adaptive algorithms to epistatic domains. pages 162–164, 1985.

Ricardo Fukasawa, Humberto Longo, Jens Lysgaard, Marcus Poggi de Aragão, Marcelo Reis, Eduardo Uchoa, and Renato F. Werneck. Robust branch-and-cut-and-price for the capacitated vehicle routing problem. *Mathematical Programming*, 106(3):491–511, 2006. doi: 10.1007/s10107-005-0644-x.

David E. Goldberg. Genetic algorithms in search, optimization and machine learning. 1989.

Bruce Golden, Edward Wasil, James Kelly, and I-Ming Chao. The impact of metaheuristics on solving the vehicle routing problem: algorithms, problem sets, and computational results. pages 33–56, 1998.

Ali Najm Jasim and Lamia Chaari Fourati. Guided genetic algorithm for solving capacitated vehicle routing problem with unmanned-aerial-vehicles. *IEEE Access*, 12:106333–106358, 2024. doi: 10.1109/ACCESS.2024.3438079.

Gilbert Laporte and Yves Nobert. A branch and bound algorithm for the capacitated vehicle routing problem. *OR Spektrum*, 5:77–85, 1983. doi: 10.1007/BF01720015.

Christian Prins. A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31(12):1985–2002, 2004. doi: 10.1016/S0305-0548(03)00158-8.

Neha Sehta and Urjita Thakar. Sweep nearest algorithm for capacitated vehicle routing problem. In *2021 IEEE 18th India Council International Conference (INDICON)*, pages 1–6, 2021. doi: 10.1109/INDICON52576.2021.9691603.

Paolo Toth and Daniele Vigo. The vehicle routing problem. 2002. doi: 10.1137/1.9780898718515.